

> parking - data

	Distance	Time of Day	Vehicle Density	Parking Position	Status
1	50	9	20	30	occupied
2	120	14	60	70	occupied
3	80	11	45	60	occupied
4	30	8	15	20	Vacant
5	200	18	80	120	vacant
6	60	16	30	45	occupied
7	150	10	70	100	occupied
8	40	7	10	75	Vacant
9	100	13	55	70	vacant
10	170	19	75	110	Vacant

Ex:No: 5A
Date: 09/04/26

Smart Parking Slot Occupancy Classification Using K-Nearest Neighbor Algorithm

Problem Description:

In a smart city environment, parking management systems collect sensor data to determine whether parking slots are **Occupied** or **Vacant**. Each parking record includes numerical attributes such as **Distance from entry gate**, **Time of day**, **Vehicle density**, and **Parking duration**.

The objective is to classify parking slot status using the **K-Nearest Neighbor (KNN)** algorithm and visualize the classification results for efficient parking management.

Objective:

To implement and visualize the K-Nearest Neighbor algorithm in R for classifying parking slot occupancy based on sensor data.

Procedure:

- Step 1: Create Parking Dataset

	Distance	TimeOfDay	VehicleDensity	ParkingDuration	Status
1	50	9	20	30	occupied
2	120	14	60	90	occupied
3	80	11	45	60	occupied
4	30	8	15	20	Vacant
5	200	18	80	120	occupied
6	60	10	30	45	Vacant
7	150	16	70	100	occupied
8	40	7	10	15	Vacant
9	100	13	55	70	occupied
10	170	19	75	110	occupied

```
Parking-data <- data.frame (
```

```
Distance = c ( 50, 120, 80, 30, 200, 60, 150, 40, 100, 170 ) ,
```

```
TimeOfDay = c ( 9, 14, 11, 8, 18, 10, 16, 7, 13, 19 ) ,
```

```
VehicleDensity = c ( 20, 60, 45, 15, 80, 30, 70, 10, 55, 75 ) ,
```

```
ParkingDuration = c ( 30, 90, 60, 20, 120, 45, 100, 15, 70, 110 ) ,
```

```
Status = c ( "occupied", "occupied", "occupied", "Vacant", "occupied",
```

```
"Vacant", "Occupied", "Vacant", "Occupied", "Occupied" )
```

- Step 2: Select Features and Class Label

features <- parking_data[, c("TimeOfDay", "VehicleDensity")]

labels <- parking_data\$status

- Step 3: Data Visualization (Scatter Plot) - TimeOfDay and VehicleDensity,

plot(features\$TimeOfDay, features\$VehicleDensity,

col = ifelse(labels == "occupied", "red", "green"),

pch = 19,

xlab = "Time Of Day",

ylab = "Vehicle Density",

main = "Scatter plot")

legend("topleft", legend = c("occupied", "vacant"),

col = c("red", "green"), pch = 19)

- Step 4: Normalize the Data

normalize <- function(x) {

(x - min(x)) / (max(x) - min(x)) }

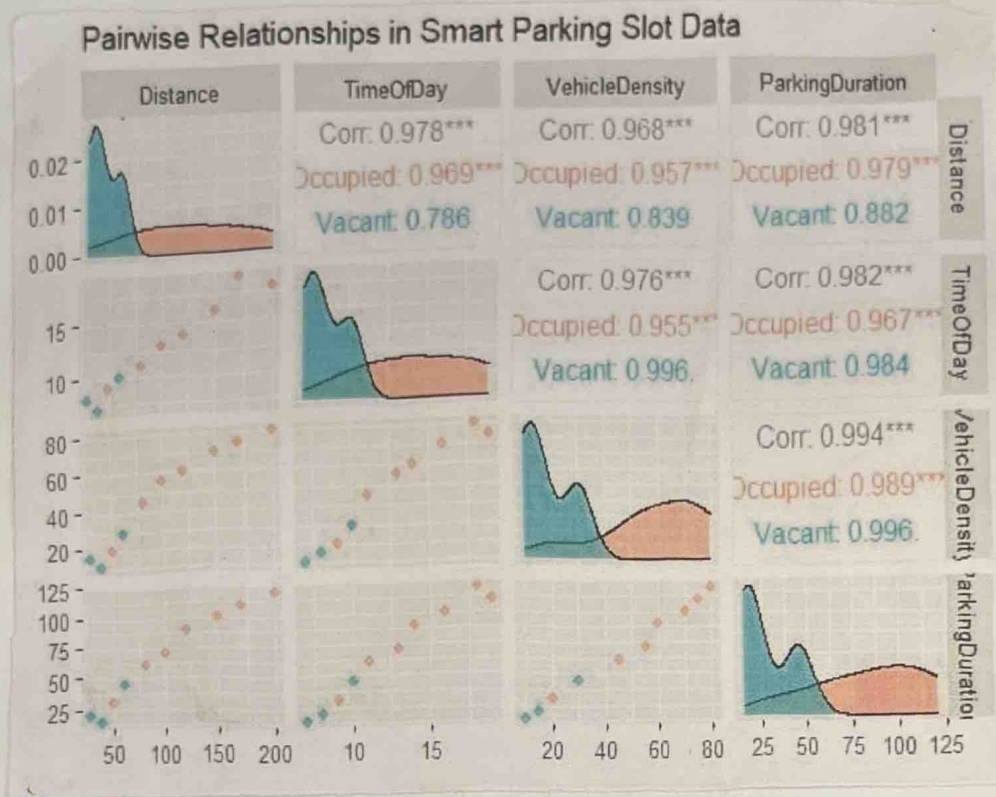
features_norm <- as.data.frame(apply(features, normalize))

features_norm

> predicted table

[1] vacant occupied occupied

Levels: occupied vacant



> confusion matrix

	Predicted	
	occupied	vacant
Actual		
occupied	2	0
vacant	0	1

> accuracy

[1]

- Step 5: Train(70%) – Test Split(30%)

```
set.seed(123)
```

```
sample_index <- sample(1:nrow(features_norm), 0.7 * nrow(features_norm))
```

```
train_data <- features_norm[sample_index, ]
```

```
test_data <- features_norm[-sample_index, ]
```

```
train_labels <- labels[sample_index]
```

```
test_labels <- labels[-sample_index]
```

- Step 6: Apply KNN Algorithm – k=3 and predict labels

```
install.packages("class")
```

```
library(class)
```

```
predicted_labels <- knn(train=train_data, test=test_data,
```

```
cl=train_labels, k=3)
```

- Step 7: Confusion Matrix and Accuracy

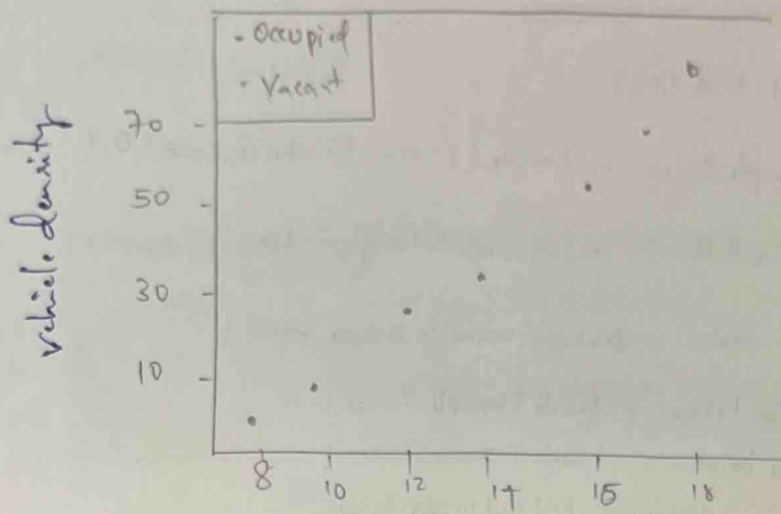
```
conf_mat <- table(predicted=predicted_labels, actual=test_labels)
```

```
conf_mat
```

```
accuracy <- sum(diag(conf_mat)) / sum(conf_mat)
```

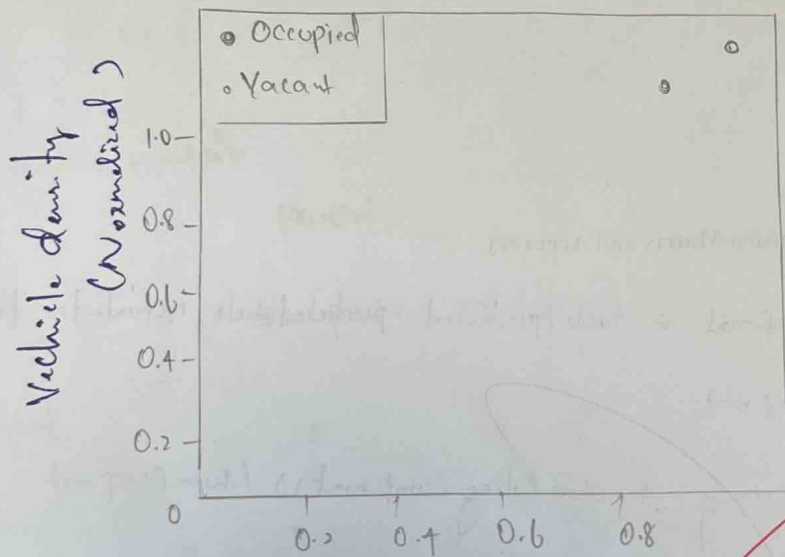
```
accuracy
```


Park slot status Distribution



Time of Day (Hours)

Knn classification Result for Parking slot



Time of Day (normalized)

- Step 8: Visualization of Classification Result – Normalized data - TimeOfDay and VehicleDensity,

```
plot (test-data$TimeOfDay, test-data$VehicleDensity,
      col = ifelse (predicted-labels == "Occupied", "red", "green"),
      pch = 19,
      xlab = "Time of Day (Normalized)",
      ylab = "Vehicle Density (Normalized)",
      main = "KNN Classification Result",
      legend ("topleft", legend = c("Occupied", "Vacant"), col = c("red", "green"), pch = 19))
```

- Step 8: Pairwise Visualization using ggpairs: To visualize relationships between multiple numerical attributes and observe how they vary with parking slot status.

```
install.packages ("Ggally")
library (Ggally)
ggpairs (parking-data, aes (color = status))
```

Result:

Thus, the program successfully demonstrates the use of the K-Nearest Neighbor algorithm in R to classify smart parking slot occupancy. The visualizations effectively represent parking patterns, and the model provides accurate classification suitable for smart city traffic and parking management systems.

Ex:No: 5B
Date: 8/04/21

Visualization and Classification of Iris Flower Species Using K-Nearest Neighbor Algorithm

Problem Description:

The Iris dataset is a well-known dataset in machine learning that contains measurements of iris flowers belonging to three different species: *Setosa*, *Versicolor*, and *Virginica*. Each flower is described using four numerical attributes: **Sepal Length**, **Sepal Width**, **Petal Length**, and **Petal Width**.

The objective is to visualize the dataset, apply the K-Nearest Neighbor (KNN) algorithm, and analyze the classification results.

Objective:

To visualize the Iris dataset and implement the K-Nearest Neighbor algorithm in R for classifying iris flower species based on their measurements.

Procedure:

- **Step 1: Install and Load Required Libraries**

We install and load the necessary libraries for implementing the K-NN algorithm.

```
# Install necessary libraries
```

```
install.packages("class")
```

```
install.packages("kKally")
```

```
install.packages("ggplot2")
```

```
# Load the libraries
```

```
library(class)
```

```
library(kKally)
```

```
library(ggplot2)
```

- **Step 2: Load and Prepare the Dataset**

We use the iris dataset for classification and check its structure.

```
data(iris)
```

```
str(iris)
```

```
str(iris)
```


Normalize the numerical data (Feature Scaling)

```
normalize <- function(x) {  
  (x - min(x)) / (max(x) - min(x))  
}
```

Apply normalization to numerical columns

```
iris_norm <- as.data.frame(lapply(iris[,1:4], normalize))
```

```
iris_norm$species <- iris$species
```

- Step 3: Split the Dataset into Training and Testing Sets

We split the dataset into 80% training and 20% testing.

```
set.seed(123)
```

```
train_index <- sample(1:nrow(iris_norm), 0.8 * nrow(iris_norm))
```

```
train_data <- iris_norm[train_index, 1:4]
```

```
test_data <- iris_norm[-train_index, 1:4]
```

Extract labels (target variable)

```
train_labels <- iris_norm$species[train_index]
```

```
test_labels <- iris_norm$species[-train_index]
```

> Knn-prediction

```

[13] setosa setosa setosa setosa setosa setosa
      setosa setosa
[13] versicol versicol versicol versicol versicol
      versicol versicol
[25] virginica virginica virginica versicol virginica
Levels: setosa versicol virginica

```

> print (Conf-matrix)

Confusion matrix and statistics

prediction	Reference		
	setosa	versicol	virginica
Setosa	10	0	0
versicol	0	10	2
virginica	0	0	8

Overall statistics

Accuracy : 0.9333

95% CI : (0.7713, 0.9918)

: 0.3333

0.89472 - 12

Kappa : 0.9

- **Step 4: Implement the K-Nearest Neighbor Algorithm**

We use knn() function to build the model.

k <- 5 # Choose number of neighbors

k <- 5

predicted_labels <- knn (train = train_data,

test = test_data,

cl = train_labels,

k = k)

- **Step 5: Evaluate Model Performance**

We compute accuracy using a confusion matrix.

Conf-mat <- table (predicted = predicted_labels, Actual = test_labels)

accuracy <- sum (diagonal (conf-mat)) / sum (conf-mat)

accuracy

- **Step 6: Visualize the Classification Results**

We plot a scatter plot of Sepal.Length vs Sepal.Width with predicted class labels.

Scatter Plot: Sepal Length vs Sepal Width, colored by Species

Plot (test_data\$sepal.length, test_data\$species\$sepal.width,

col = as.numeric (predicted_labels),

pch = 19,

xlab = "sepal.length",

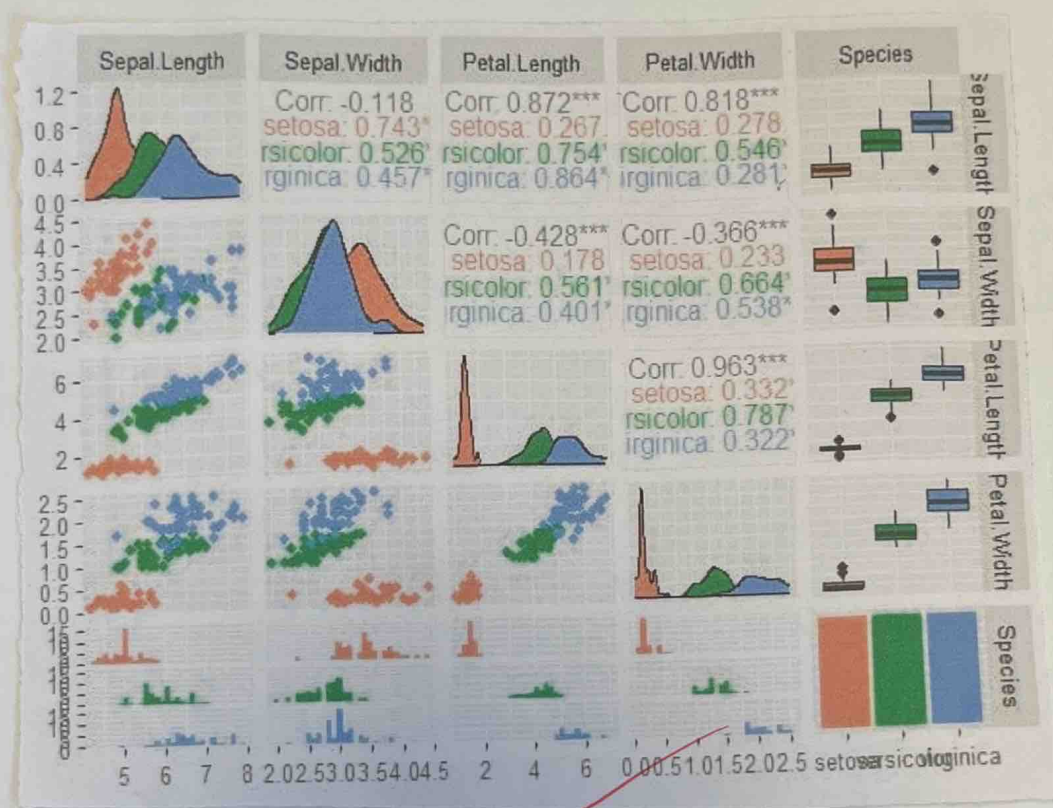
ylab = "sepal width"

main = "knn classification")

legend ("topright", legend = levels (predicted_labels), col = 1:length (

Statistics by class:

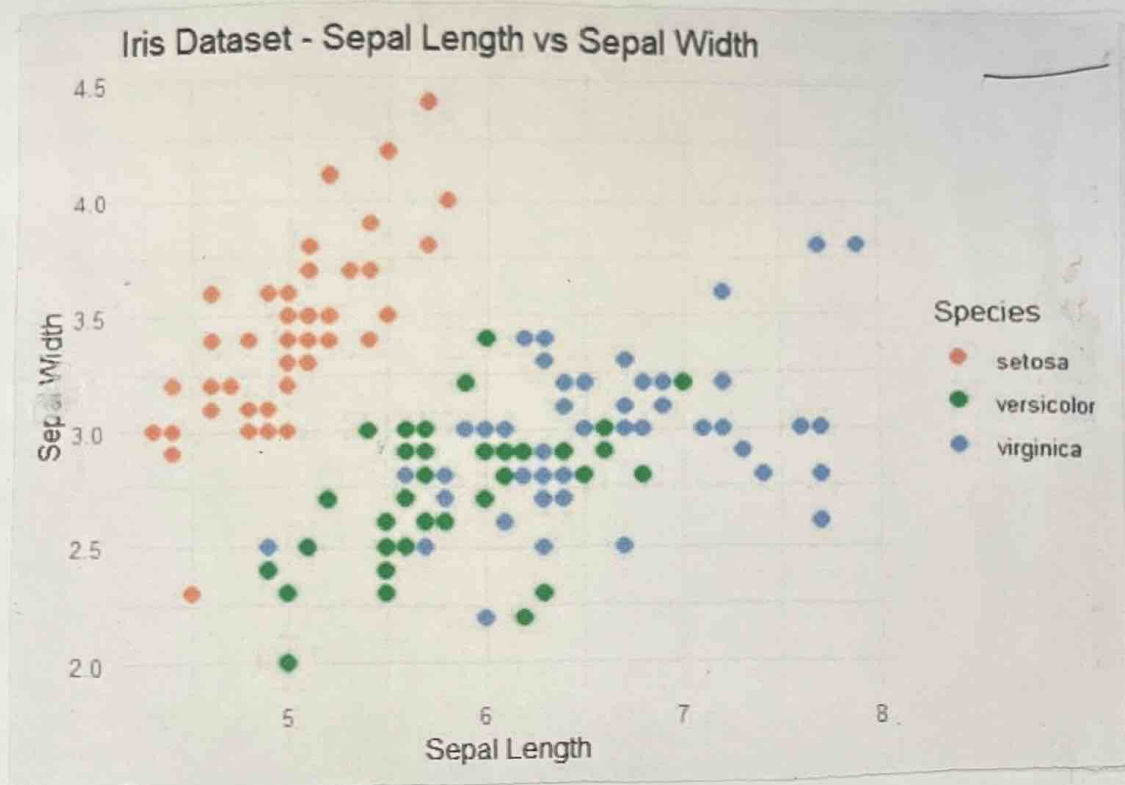
	class: setosa	class: versicol	class: virginica
Sensitivity	1.000	1.000	0.600
Specificity	1.000	0.900	1.000
Pos pred value	1.000	0.833	0.709
Neg pred value	1.000	1.000	0.333
Detection Rate	0.333	0.333	0.267
prevalence	0.333	0.333	0.206
Detection prevalence	0.333	0.400	0.100
Balanced Accuracy	1.000	0.950	0.100



Now, we generate a pair plot using the `ggpairs()` function, which visualizes the relationships between all numeric features.

Pair Plot: Relationship between all numeric features

```
ggpairs(iris, aes(color = species))  
ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, color = species)) +  
  geom_point(size = 3) + labs(title = "Iris Dataset - Sepal Length vs Sepal  
    width", x = "sepal length", y = "sepal width") + theme_minimal()
```



Result:

Thus, the program successfully demonstrates the visualization and classification of the Iris dataset using the K-Nearest Neighbor algorithm in R. The visualizations reveal clear class separations, and the KNN model achieves high classification accuracy, confirming its effectiveness for multi-class classification problems.